CS327: Compilers

Assignment 3: Syntax Analyzer

Group Members:
Gaurav Budhwani (22110085)
John Twipraham Debbarma (22110108)
Harshit (22110095)

March 15, 2026

Contents

1 Introduction

This report documents the implementation of the Syntax Analyzer for the **OPEX** programming language. The parser uses **Bison** (Yacc) to define and enforce the Context-Free Grammar (CFG), and integrates with **Flex** for lexical analysis. It successfully parses 12 diverse test cases ranging from basic graph algorithms to computer vision pipelines.

2 Context-Free Grammar (CFG)

The following non-exhaustive list of grammar rules illustrates the core structure of the syntax:

```
Program     -> PROGRAM ID Declarations MainBlock
MainBlock   -> MAIN BEGIN Statements END | BEGIN Statements END
Declarations-> /* empty */ | Declarations Decl
Decl        -> VarDecl | StructDecl | FuncDecl
VarDecl     -> VAR ID : Type OptSemi
             | AUTO ID := Expr OptSemi
             | Type ID := Expr OptSemi
Type        -> INT | FLOAT | BOOL | STRING | VOID | TYPE_CV | ArrayType | ID
StructDecl  -> STRUCT ID StructMembers ENDSTRUCT OptSemi
FuncDecl    -> FUNCTION ID ( ParamList ) : Type BEGIN Statements END OptSemi
Statements  -> /* empty */ | Statements Statement
Statement   -> AssignStmt | IfStmt | WhileStmt | ForStmt | ForeachStmt
             | MatchStmt | IOStmt | ReturnStmt | FuncCallStmt OptSemi
AssignStmt  -> LValue := Expr OptSemi | LValue += Expr OptSemi | LValue ++ OptSemi
IOStmt      -> PRINT Expr OptSemi | INPUT LValue OptSemi
```

3 Source Code

3.1 parser.y

The Bison specification that defines the OPEX grammar rules.

```
%{
#include <stdio.h>
#include <stdlib.h>

extern int yylex();
extern int yyparse();
extern int line_num;

void yyerror(const char *s) {
    fprintf(stderr, "Syntax Error at line %d: %s\n", line_num, s);
    exit(1);
}
%}

// Keywords
%token TOK_PROGRAM TOK_BEGIN TOK_END TOK_FUNCTION TOK_RETURN TOK_MAIN
%token TOK_STRUCT TOK_ENDSTRUCT TOK_VAR TOK_AUTO TOK_ARRAY TOK_OF
```

```
%token TOK_INT TOK_FLOAT TOK_BOOL TOK_STRING TOK_VOID
%token TOK_IMAGE TOK_MATRIX TOK_KERNEL TOK_PIXEL
%token TOK_MATCH TOK_WHEN TOK_ENDMATCH TOK_IF TOK_THEN TOK_ELSE
    TOK_ENDIF
%token TOK_FOR TOK_FOREACH TOK_IN TOK_TO TOK_DO TOK_ENDFOR TOK_STEP
    TOK_WHILE TOK_ENDWHILE
%token TOK_PRINT TOK_INPUT TOK_TRUE TOK_FALSE TOK_AND TOK_OR TOK_NOT
%token TOK_LOAD TOK_SAVE TOK_GRAYSCALE TOK_HSV TOK_BLUR TOK_THRESHOLD
    TOK_CANNY TOK_RESIZE TOK_ROTATE TOK_ERODE TOK_DILATE TOK_FLIP

// Operators
%token TOK_ASSIGN TOK_RANGE TOK_INC TOK_DEC
%token TOK_PLUS_ASSIGN TOK_MINUS_ASSIGN TOK_MULT_ASSIGN TOK_DIV_ASSIGN
%token TOK_CONV_OP TOK_PIPE_OP
%token TOK_PLUS TOK_MINUS TOK_MULT TOK_DIV TOK_MOD
%token TOK_EQ TOK_NEQ TOK_LEQ TOK_GEQ TOK_LT TOK_GT

// Separators
%token TOK_LBRACKET TOK_RBRACKET TOK_LPAREN TOK_RPAREN TOK_COLON
    TOK_COMMA TOK_DOT TOK_SEMI

// Literals and Identifiers
%token TOK_ID TOK_HEX_INT TOK_FLOAT_LIT TOK_INT_LIT TOK_STR

// Precedence and Associativity
%right TOK_ASSIGN TOK_PLUS_ASSIGN TOK_MINUS_ASSIGN TOK_MULT_ASSIGN
    TOK_DIV_ASSIGN
%left TOK_OR
%left TOK_AND
%left TOK_EQ TOK_NEQ TOK_LEQ TOK_GEQ TOK_LT TOK_GT
%left TOK_PLUS TOK_MINUS
%left TOK_MULT TOK_DIV TOK_MOD
%right TOK_CONV_OP TOK_PIPE_OP
%right TOK_NOT
%left TOK_LBRACKET TOK_RBRACKET TOK_LPAREN TOK_RPAREN TOK_DOT

%start Program

%%

Program:
    TOK_PROGRAM TOK_ID Declarations MainBlock
    | TOK_PROGRAM TOK_ID MainBlock
    | TOK_PROGRAM TOK_ID Declarations
    ;

Declarations:
    Decl
    | Declarations Decl
    ;

Decl:
    VarDecl
    | StructDecl
    | FuncDecl
    ;

Type:
```

```
71      TOK_INT | TOK_FLOAT | TOK_BOOL | TOK_STRING | TOK_VOID
72      | TOK_IMAGE | TOK_MATRIX | TOK_KERNEL | TOK_PIXEL
73      | ArrayType
74      | TOK_ID
75      ;
76
77  ArrayType:
78      TOK_ARRAY TOK_LBRACKET TOK_INT_LIT TOK_RBRACKET TOK_OF Type
79      | TOK_ARRAY TOK_LBRACKET Expr TOK_RBRACKET TOK_OF Type
80      | TOK_ARRAY TOK_LBRACKET TOK_INT_LIT TOK_RBRACKET TOK_LBRACKET
        TOK_INT_LIT TOK_RBRACKET TOK_OF Type
81      | TOK_ARRAY TOK_LBRACKET Expr TOK_RBRACKET TOK_LBRACKET Expr
        TOK_RBRACKET TOK_OF Type
82      ;
83
84  OptSemi:
85      TOK_SEMI
86      | /* empty */
87      ;
88
89  VarDecl:
90      TOK_VAR TOK_ID TOK_COLON Type OptSemi
91      | TOK_AUTO TOK_ID TOK_ASSIGN Expr OptSemi
92      | Type TOK_ID TOK_ASSIGN Expr OptSemi
93      ;
94
95  StructDecl:
96      TOK_STRUCT TOK_ID StructMembers TOK_ENDSTRUCT OptSemi
97      | TOK_STRUCT TOK_ID TOK_ENDSTRUCT OptSemi
98      ;
99
100 StructMembers:
101     StructMember
102     | StructMembers StructMember
103     ;
104
105 StructMember:
106     TOK_ID TOK_COLON Type OptSemi
107     ;
108
109 FuncDecl:
110     TOK_FUNCTION TOK_ID TOK_LPAREN ParamList TOK_RPAREN TOK_COLON Type
        TOK_BEGIN Statements TOK_END OptSemi
111     | TOK_FUNCTION TOK_ID TOK_LPAREN TOK_RPAREN TOK_COLON Type
        TOK_BEGIN Statements TOK_END OptSemi
112     | TOK_FUNCTION TOK_ID TOK_LPAREN ParamList TOK_RPAREN TOK_COLON
        Type TOK_BEGIN TOK_END OptSemi
113     | TOK_FUNCTION TOK_ID TOK_LPAREN TOK_RPAREN TOK_COLON Type
        TOK_BEGIN TOK_END OptSemi
114     | TOK_FUNCTION TOK_ID TOK_LPAREN ParamList TOK_RPAREN TOK_COLON
        Type MainBlock OptSemi
115     | TOK_FUNCTION TOK_ID TOK_LPAREN TOK_RPAREN TOK_COLON Type
        MainBlock OptSemi
116     ;
117
118 ParamList:
119     Param
120     | ParamList TOK_COMMA Param
```

```
    ;

Param:
    TOK_ID TOK_COLON Type
    ;

MainBlock:
    TOK_MAIN TOK_BEGIN Statements TOK_END OptSemi
    | TOK_BEGIN Statements TOK_END OptSemi
    | TOK_MAIN TOK_BEGIN TOK_END OptSemi
    | TOK_BEGIN TOK_END OptSemi
    ;

Statements:
    Statement
    | Statements Statement
    ;

Statement:
    VarDecl
    | AssignStmt
    | IfStmt
    | WhileStmt
    | ForStmt
    | ForeachStmt
    | MatchStmt
    | IOStmt
    | ReturnStmt
    | FuncCallStmt OptSemi
    | Expr OptSemi
    ;

AssignStmt:
    LValue TOK_ASSIGN Expr OptSemi
    | LValue TOK_PLUS_ASSIGN Expr OptSemi
    | LValue TOK_MINUS_ASSIGN Expr OptSemi
    | LValue TOK_MULT_ASSIGN Expr OptSemi
    | LValue TOK_DIV_ASSIGN Expr OptSemi
    | LValue TOK_INC OptSemi
    | LValue TOK_DEC OptSemi
    ;

LValue:
    TOK_ID
    | LValue TOK_LBRACKET Expr TOK_RBRACKET
    | LValue TOK_DOT TOK_ID
    ;

IfStmt:
    TOK_IF Expr TOK_THEN Statements TOK_ENDIF OptSemi
    | TOK_IF Expr TOK_THEN Statements TOK_ELSE Statements TOK_ENDIF
   OptSemi
    | TOK_IF Expr TOK_THEN TOK_ENDIF OptSemi
    | TOK_IF Expr TOK_THEN TOK_ELSE Statements TOK_ENDIF OptSemi
    | TOK_IF Expr TOK_THEN Statements TOK_ELSE TOK_ENDIF OptSemi
    ;

WhileStmt:
```

```
178     TOK_WHILE Expr TOK_DO Statements TOK_ENDWHILE OptSemi
179     | TOK_WHILE Expr TOK_DO TOK_ENDWHILE OptSemi
180     ;
181
182 ForStmt:
183     TOK_FOR LValue TOK_ASSIGN Expr TOK_TO Expr TOK_DO Statements
    TOK_ENDFOR OptSemi
184     | TOK_FOR LValue TOK_ASSIGN Expr TOK_TO Expr TOK_DO TOK_ENDFOR
    OptSemi
185     | TOK_FOR LValue TOK_ASSIGN Expr TOK_TO Expr TOK_STEP Expr TOK_DO
    Statements TOK_ENDFOR OptSemi
186     | TOK_FOR LValue TOK_ASSIGN Expr TOK_TO Expr TOK_STEP Expr TOK_DO
    TOK_ENDFOR OptSemi
187     ;
188
189 ForeachStmt:
190     TOK_FOREACH TOK_ID TOK_IN Expr TOK_DO Statements TOK_ENDFOR OptSemi
191     | TOK_FOREACH TOK_ID TOK_IN Expr TOK_DO TOK_ENDFOR OptSemi
192     ;
193
194 MatchStmt:
195     TOK_MATCH Expr TOK_WHEN Expr TOK_COLON Statements TOK_ENDMATCH
    OptSemi
196     | TOK_MATCH Expr MatchWhenList TOK_ENDMATCH OptSemi
197     ;
198
199 MatchWhenList:
200     MatchWhen
201     | MatchWhenList MatchWhen
202     ;
203
204 MatchWhen:
205     TOK_WHEN Expr TOK_COLON Statements
206     | TOK_WHEN Expr TOK_COLON
207     ;
208
209 IOStmt:
210     TOK_PRINT Expr OptSemi
211     | TOK_PRINT TOK_STR OptSemi
212     | TOK_INPUT LValue OptSemi
213     ;
214
215 ReturnStmt:
216     TOK_RETURN Expr OptSemi
217     | TOK_RETURN OptSemi
218     ;
219
220 FuncCallStmt:
221     TOK_ID TOK_LPAREN ArgList TOK_RPAREN
222     | TOK_ID TOK_LPAREN TOK_RPAREN
223     | ImageOpFunc
224     ;
225
226 ImageOpFunc:
227     TOK_LOAD TOK_LPAREN ArgList TOK_RPAREN
228     | TOK_SAVE TOK_LPAREN ArgList TOK_RPAREN
229     | TOK_GRAYSCALE TOK_LPAREN ArgList TOK_RPAREN
230     | TOK_HSV TOK_LPAREN ArgList TOK_RPAREN
```

```
    | TOK_BLUR TOK_LPAREN ArgList TOK_RPAREN
    | TOK_THRESHOLD TOK_LPAREN ArgList TOK_RPAREN
    | TOK_CANNY TOK_LPAREN ArgList TOK_RPAREN
    | TOK_RESIZE TOK_LPAREN ArgList TOK_RPAREN
    | TOK_ROTATE TOK_LPAREN ArgList TOK_RPAREN
    | TOK_ERODE TOK_LPAREN ArgList TOK_RPAREN
    | TOK_DILATE TOK_LPAREN ArgList TOK_RPAREN
    | TOK_FLIP TOK_LPAREN ArgList TOK_RPAREN
    ;

ArgList:
    Expr
    | ArgList TOK_COMMA Expr
    ;

Expr:
    Expr TOK_PLUS Expr
    | Expr TOK_MINUS Expr
    | Expr TOK_MULT Expr
    | Expr TOK_DIV Expr
    | Expr TOK_MOD Expr
    | Expr TOK_EQ Expr
    | Expr TOK_NEQ Expr
    | Expr TOK_LEQ Expr
    | Expr TOK_GEQ Expr
    | Expr TOK_LT Expr
    | Expr TOK_GT Expr
    | Expr TOK_AND Expr
    | Expr TOK_OR Expr
    | Expr TOK_CONV_OP Expr
    | Expr TOK_PIPE_OP Expr
    | TOK_NOT Expr
    | TOK_MINUS Expr %prec TOK_MULT
    | TOK_LPAREN Expr TOK_RPAREN
    | LValue
    | FuncCallStmt
    | TOK_INT_LIT
    | TOK_FLOAT_LIT
    | TOK_HEX_INT
    | TOK_TRUE
    | TOK_FALSE
    | TOK_STR
    | RangeExpr
    | ArrayInitializer
    ;

RangeExpr:
    Expr TOK_RANGE Expr
    ;

ArrayInitializer:
    TOK_LBRACKET ArgList TOK_RBRACKET
    | TOK_LBRACKET TOK_RBRACKET
    ;

%%

int main(void) {
```

```
    if (yyparse() == 0) {
        printf("Syntax Correct\n");
    }
    return 0;
}
```

Listing 1: parser.y

3.2 lexer.l

The Flex lexer was modified to return Bison tokens instead of printing them out directly.

```
%option noyywrap case-sensitive
%{
#include <stdio.h>
#include "parser.tab.h"
int line_num = 1;
%}

/* DEFINITIONS */
DIGIT       [0-9]
LETTER      [a-zA-Z]
ID          {LETTER}({LETTER}|{DIGIT}|_)*
NUMBER      {DIGIT}+(\.{DIGIT}+)?
STRING      \"([^\"\\\n]|\\.)*\"
HEXINT      0[xX][0-9a-fA-F]+
FLOAT       [0-9]+\.[0-9]+([eE][+-]?[0-9]+)?
INTEGER     [0-9]+

%%
"//".*                          { /* Ignore Single line */ }
"/*"([^*]|\*+[^*/])*\*+"/"      { /* Ignore Block comment */ }
[ \t\r]                         { /* Ignore whitespace */ }
\n                              { line_num++; }

    /* KEYWORDS */
"PROGRAM"                       { return TOK_PROGRAM; }
"BEGIN"                         { return TOK_BEGIN; }
"END"                           { return TOK_END; }
"FUNCTION"                      { return TOK_FUNCTION; }
"RETURN"                        { return TOK_RETURN; }
"MAIN"                          { return TOK_MAIN; }
"STRUCT"                        { return TOK_STRUCT; }
"ENDSTRUCT"                     { return TOK_ENDSTRUCT; }
"VAR"                           { return TOK_VAR; }
"AUTO"                          { return TOK_AUTO; }
"ARRAY"                         { return TOK_ARRAY; }
"OF"                            { return TOK_OF; }

    /* TYPES */
"INT"                           { return TOK_INT; }
"FLOAT"                         { return TOK_FLOAT; }
"BOOL"                          { return TOK_BOOL; }
"STRING"                        { return TOK_STRING; }
"VOID"                          { return TOK_VOID; }
"IMAGE"                         { return TOK_IMAGE; }
"MATRIX"                        { return TOK_MATRIX; }
"KERNEL"                        { return TOK_KERNEL; }
```

```
"PIXEL"                     { return TOK_PIXEL; }

    /* CONTROL FLOW */
"MATCH"                     { return TOK_MATCH; }
"WHEN"                      { return TOK_WHEN; }
"ENDMATCH"                  { return TOK_ENDMATCH; }
"IF"                        { return TOK_IF; }
"THEN"                      { return TOK_THEN; }
"ELSE"                      { return TOK_ELSE; }
"ENDIF"                     { return TOK_ENDIF; }
"FOR"                       { return TOK_FOR; }
"FOREACH"                   { return TOK_FOREACH; }
"IN"                        { return TOK_IN; }
"WHILE"                     { return TOK_WHILE; }
"ENDWHILE"                  { return TOK_ENDWHILE; }
"TO"                        { return TOK_TO; }
"DO"                        { return TOK_DO; }
"ENDFOR"                    { return TOK_ENDFOR; }
"STEP"                      { return TOK_STEP; }

    /* I/O AND BUILT-INS */
"PRINT"                     { return TOK_PRINT; }
"INPUT"                     { return TOK_INPUT; }
"TRUE"                      { return TOK_TRUE; }
"FALSE"                     { return TOK_FALSE; }
"LOAD"                      { return TOK_LOAD; }
"SAVE"                      { return TOK_SAVE; }
"GRAYSCALE"                 { return TOK_GRAYSCALE; }
"HSV"                       { return TOK_HSV; }
"BLUR"                      { return TOK_BLUR; }
"THRESHOLD"                 { return TOK_THRESHOLD; }
"CANNY"                     { return TOK_CANNY; }
"RESIZE"                    { return TOK_RESIZE; }
"ROTATE"                    { return TOK_ROTATE; }
"ERODE"                     { return TOK_ERODE; }
"DILATE"                    { return TOK_DILATE; }
"FLIP"                      { return TOK_FLIP; }

    /* LOGICAL OPERATORS */
"AND"                       { return TOK_AND; }
"OR"                        { return TOK_OR; }
"NOT"                       { return TOK_NOT; }

    /* OPERATORS */
":="                        { return TOK_ASSIGN; }
".."                        { return TOK_RANGE; }
"++"                        { return TOK_INC; }
"--"                        { return TOK_DEC; }
"+="                        { return TOK_PLUS_ASSIGN; }
"-="                        { return TOK_MINUS_ASSIGN; }
"*="                        { return TOK_MULT_ASSIGN; }
"/="                        { return TOK_DIV_ASSIGN; }
"**"                        { return TOK_CONV_OP; }
"|>"                        { return TOK_PIPE_OP; }
"+"                         { return TOK_PLUS; }
"-"                         { return TOK_MINUS; }
"*"                         { return TOK_MULT; }
"/"                         { return TOK_DIV; }
```

```
"%"                             { return TOK_MOD; }
"=="                            { return TOK_EQ; }
"!="                            { return TOK_NEQ; }
"<="                            { return TOK_LEQ; }
">="                            { return TOK_GEQ; }
"<"                             { return TOK_LT; }
">"                             { return TOK_GT; }

    /* SEPARATORS */
"["                             { return TOK_LBRACKET; }
"]"                             { return TOK_RBRACKET; }
"("                             { return TOK_LPAREN; }
")"                             { return TOK_RPAREN; }
":"                             { return TOK_COLON; }
","                             { return TOK_COMMA; }
"."                             { return TOK_DOT; }
";"                             { return TOK_SEMI; }

    /* LITERALS AND IDENTIFIERS */
{ID}                            { return TOK_ID; }
{HEXINT}                        { return TOK_HEX_INT; }
{FLOAT}                         { return TOK_FLOAT_LIT; }
{INTEGER}                       { return TOK_INT_LIT; }
{STRING}                        { return TOK_STR; }

    /* ERROR HANDLING */
.                               { printf("ERR: %s line %d\n", yytext,
   line_num); exit(1); }
%%
```

Listing 2: lexer.l

3.3 Makefile

The build system compiles both files.

```
CC = gcc
CFLAGS = -I src
FLEX = flex
BISON = bison

all: parser

parser: src/lex.yy.c src/parser.tab.c
  $(CC) $(CFLAGS) src/lex.yy.c src/parser.tab.c -o parser

src/lex.yy.c: src/lexer.l src/parser.tab.h
  $(FLEX) -o src/lex.yy.c src/lexer.l

src/parser.tab.c src/parser.tab.h: src/parser.y
  $(BISON) -d -o src/parser.tab.c src/parser.y

test: parser
  ./parser < tests/sample.opx

test_all: parser
  @for f in tests/*.opx; do \
    echo -n "Testing $$f... "; \
```

```
    ./parser < "$$f"; \
  done

clean:
  rm -f src/lex.yy.c src/parser.tab.c src/parser.tab.h parser
```

Listing 3: Makefile

4 Test Suite Coverage

All 12 OPEX test files are available in the `tests/` directory and pass the parser evaluation correctly:

- `binary_search.opx`: Array slicing and control flows.
- `edge_detection.opx`: Custom type instantiation (`KERNEL`) and implicit declarations (`AUTO`).
- `sample.opx`: Multi-dimensional arrays and foreach loops.
- `queue.opx` & `stack.opx`: Object-struct definitions and usages.
- `floyd_warshall.opx`: Triply-nested loops and matrix accesses.
- ... and six additional algorithmic tests (`dfs.opx`, `bst_inorder.opx`, `fibonacci.opx`, etc.)